

UNIVERSITY ACADEMIC PROJECT REPORT

PhoneRadModeler: App-Specific Mobile Radiation Exposure Simulator Engine

An Offline Heuristic Simulation Model and System Architecture

Course Title: Introduction to Problem Solving and Programming in Python

Project Module: Individual Repository Evaluation Submission Track

Framework Environment: Decoupled Standard Library Python Environment

Author Target User Profiles: Single Academic Research Portfolio

Submission Portal Status: Ready for Compilation Checklist

1. Introduction

The widespread usage of modern mobile devices exposes individuals to continuously varying amounts of radiofrequency electromagnetic fields (RF-EMF). While hardware standards enforce global safe operating boundaries via Specific Absorption Rate (SAR) limits, users lack operational awareness of how running personal software applications directly impacts current antenna transmissions. This project demonstrates a programmatic methodology to simulate, validate, and track application-level electromagnetic radiation exposures locally through decoupled, test-driven Python development architectures.

2. Problem Statement

Mobile operating systems seal access to raw radio transceiver basebands beneath secure sandbox layers, preventing individual consumers from querying actual hardware transmission power metrics. Consequently, consumers remain unaware of how resource-heavy operations—such as background location-tracking tasks, heavy GPU processing, or continuous high-bandwidth 5G transmissions—affect radiation variables. **PhoneRadModeler** circumvents these hardware access blocks by building an interactive, rule-based algorithmic matrix engine that scales mathematical telemetry indicators dynamically to inform users.

3. Functional Requirements

The core software must implement at least three core functional capabilities:

FR1 - Telemetry Ingestion Perimeter Validation: The system must strictly ingest, parse, and clean application metrics (session duration, network protocol flags, and background sensor activity toggles), using explicit catch structures to route incorrect inputs.

FR2 - Heuristic SAR Multiplier Computation: The processing engine must map operational telemetry rows to weighted formulas to compute instantaneous SAR profiles bounded below regulatory maximum limits (1.60 W/kg).

FR3 - Persistent File Serialization Ledger: Calculated exposure records must append seamlessly to an offline JSON structured file, updating a regional data history accessible via the text menu system.

4. Non-Functional Requirements

The software architecture maintains stability by prioritizing these performance attributes:

NFR1 - Execution Latency Constraints: Core algorithmic transformations and calculation lookups must return final execution payloads in less than 500 milliseconds.

NFR2 - Zero-Dependency Reliability: System must build exclusively on top of the native Python Standard Library to ensure platform-agnostic portability on any operating terminal without third-party environment installation.

NFR3 - Graceful Maintainability: The platform must spread functionality across 5 to 10 decoupled modules to prevent data cross-contamination and support clean isolated maintenance.

NFR4 - High Ingestion Fault Tolerance: The application perimeter must gracefully isolate runtime casting exceptions without breaking active interaction loops or corrupting data file registries.

5. System Architecture & Folder Topography

The GitHub target deployment architecture implements clean modular separation, separating configuration constants, interactive entryways, and core mathematical transformation models:

```
phone-radiation-app-simulator/
├── data/
│   └── radiation_history.json      # Permanent historical storage ledger
├── src/
│   ├── __init__.py               # Structural package initialization identifier
│   ├── main.py                   # Main entrypoint, interface loop & route manager
│   ├── config.py                 # Scientific emission weights & baseline constants
│   └── validator.py              # Structural ingestion sanitizer & range boundary f
irewall
├── calculator.py                 # Core rule formula mapping and calculation logic
├── tests/
│   └── test_calculations.py       # Test assertion validation suite for pytest
├── README.md                    # Master repository technical blueprint documentati
on
└── statement.md                  # Domain scoping outline & user persona targets
```

6. Design Decisions & Algorithmic Rationale

To model real-world physics conditions accurately without direct hardware sensors, the algorithm calculates radiation exposure based on how different system processes interact:

Component Variable	Assigned Rule Weight / Impact	Engineering & Physical Justification
Base Baseline Asset	0.15 W/kg	Default internal idle operational radio state.
5G Networking Channel	+0.75 W/kg	Dense high-frequency millimeter-wave transmission load.
4G Networking Channel	+0.45 W/kg	Standard cellular tower long-range transceiver broadcast.
Wi-Fi Local Channel	+0.20 W/kg	Short-range localized indoor transmission loop.
Active Background GPS	+0.35 W/kg	Continuous multi-satellite array synchronization overhead.

7. Technical Implementation Modules

The operational core of the computing system relies on explicit module scripts:

src/calculator.py - Processing Logic:

```
def calculate_session_sar(duration: float, network: str, gps_active: bool) ->
tuple[float, float, str]:
    base_sar = 0.15
    network = network.strip().upper()
    network_weight = 0.75 if network == "5G" else (0.45 if network == "4G" else 0.20)
    gps_penalty = 0.35 if gps_active else 0.00
    sar_score = base_sar + network_weight + gps_penalty
    total_exposure_load = sar_score * duration
    if sar_score <= 0.45:
        threat_tier = "LOW EXPOSURE"
    elif sar_score <= 1.10:
        threat_tier = "MODERATE EXPOSURE"
    else:
        sar_score = min(sar_score, 1.60)
        threat_tier = "HIGH EXPOSURE / CRITICAL CEILING"
    return round(sar_score, 2), round(total_exposure_load, 2), threat_tier
```

8. Testing Approach & Verification Framework

The software implements testing requirements by running a suite of automated diagnostic checks. These evaluate calculation boundaries, check for edge-case errors, and ensure that data is cleanly structured before deployment:

```
def test_low_impact_app_profile():
    """Validates that short Wi-Fi runs do not spike exposure boundaries."""
    sar, load, tier = calculate_session_sar(10, "Wi-Fi", False)
    assert sar <= 0.40
    assert "LOW" in tier

def test_extreme_5g_gps_boundary():
    """Ensures that heavy network operations scale within regulatory limits."""
    sar, load, tier = calculate_session_sar(60, "5G", True)
    assert sar <= 1.60 # Guard tracking constraints strictly
    assert load > 0.0
```

9. Challenges Faced & Structural Resolutions

1. Hardware Extraction Roadblocks: Mobile operational systems prohibit direct runtime query hooks to active transceiver chips. *Resolution:* Engineered a robust heuristic simulation matrix using hardware variables to mimic system resource states accurately.

2. Ingestion Format Multiplicity: Handled instances where raw terminal inputs included varying lowercase inputs or trailing white spaces. *Resolution:* Standardized inputs using string sanitizers (`(.strip().upper())`) inside the boundary verification layers.

10. Learnings & Key Takeaways

Key educational outcomes from this project implementation include: mastering modular folder management by creating cohesive, single-responsibility files; handling execution anomalies through type-check loops to protect system operations; and implementing Git-driven version tracking to manage software rollouts cleanly.

11. Future Enhancements

The simulation ecosystem can be expanded by creating a responsive **Streamlit Web Graphical User Interface** to plot exposure trends over time, implementing background process listeners via native desktop libraries, or connecting with local networks to query current Wi-Fi data use metrics directly.

12. References

[1] Federal Communications Commission (FCC). *Mobile Device RF Exposure and Specific Absorption Rate Standards*. IEEE Guidelines Regulation Metric Core documentation.

[2] Python Software Foundation. *The Python Standard Library Documentation Reference (Modules: JSON, OS, Datetime)*. Distributed via docs.python.org.

[3] World Health Organization (WHO). *Electromagnetic Fields and Public Health: Mobile Phones Telemetry Tracking Research Studies*. Technical Document Publication Archive.